Monte Carlo methods

Craig McNeile
craig.mcneile@plymouth.ac.uk

School of Engineering, Computing and Mathematics

December 2, 2025

Introduction to the Monte Carlo method

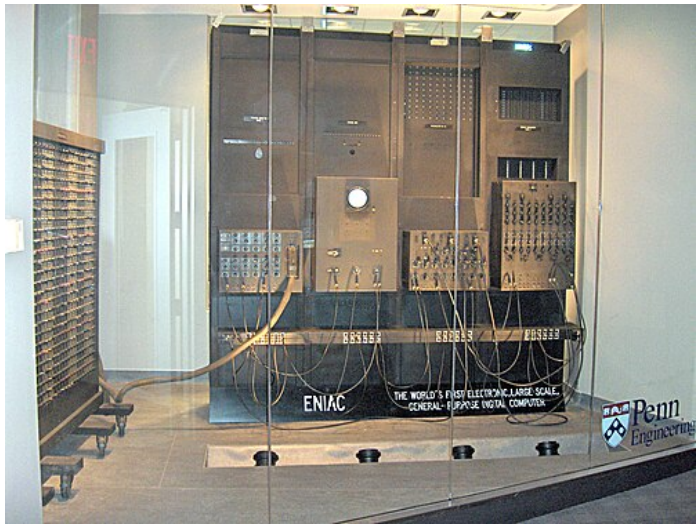
- Essentially the use of random variables to solve physical problems.
- Also sometimes known as simulation.
- The Monte Carlo method can be used to solve problems with no analytic solution, but it is also a simpler method than analytic solution.

Overview of the contents

- Random variable generation.
- Monte Carlo integration,
- Variance reduction techniques:
  - importance sampling.
  - Stratified sampling
- Simulation of Physics models.
- Simulation of gambling.
- Monte Carlo simulations of Physical Systems.
- Generation of random numbers.
- Solving a Partial Differential Equation (PDE) using MC.
- Queuing systems.

History of Monte Carlo Methods

- Developed by Stanisław Ulam at Los Alamos National Laboratory while working on the theory behind Nuclear weapons.
- ENIAC (Electronic Numerical Integrator and Computer) built in 1945
- First used to calculate neutron diffusion paths for the hydrogen bomb.



Multi-dimensional integrals

1 D integral $\int_0^1 f(x)dx$

2 D integral $\int_0^1 \int_0^1 f(x,y)dxdy$

3 D integral $\int_0^1 \int_0^1 \int_0^1 f(x,y,z)dxdydz$

Multidimensional integrals are important in many applications such as finance, particle physics physics, Bayesian statistics.

The moment of inertia of a continuous body rotating about a specified axis

$$I_P = \int \int \int_Q \rho(x,y,z) \mid r \mid^2 dV$$

Numerical integration

One dimensional integrals can be computed numerically with rules such as the trapezoidal rule.

$$\int_0^1 f(x)dx \approx \frac{f(0)+f(1)}{2n} + \frac{1}{n}\sum_{i=1}^{n-1} f(i/n)$$

There are more sophisticated methods to numerically compute an integral. Matlab can intergrate using a function "fun" using

```
q = integral(fun,xmin,xmax)
fun = @(x) exp(-x.^2).*log(x).^2;
```

Very accurate answers can be calculated quickly with errors under 10^{-5}.

Numerical integration in multidimensions

Matlab has functions to compute integrals in 2 and 3 dimensions.

- **integral2** compute 2D intergrals.
- **integral3** compute 3D intergrals.

As the number of dimensions increases a larger number of grid points is needed.

- The error on integration from the trapezoidal rule goes like $O(n^{-2/d})$ where d is the dimension of the integral.

Better algorithms are needed for higher dimensions integrals.

Monte Carlo integration

The integral α

$$\alpha = \int_0^1 f(x)dx$$

can be interpreted as the expectation $E[f(U)]$ where U uniformly drawn from [0,1].

$$\hat{\alpha}_n = \frac{1}{n}\sum_{i=1}^{n} f(U_i)$$

If the function f is integral over the region 0 to 1, then by the law of strong numbers

$$\hat{\alpha}_n \to \alpha \text{ with probability 1 as } n \to \infty$$

Error estimates of Monte Carlo integration

$$\sigma_f^2 = \int_0^1 (f(x) - \alpha)^2 dx$$

- The error on $\alpha_n - \alpha$ in the Monte Carlo estimate is approximately normal distributed with mean 0 and standard deviation $\frac{\sigma_f}{\sqrt{n}}$
- As n gets bigger then the error is reduced.

Normally σ_f is unknown, but it can estimated from the sample standard deviation.

$$s_f = \sqrt{\frac{1}{n-1} \sum_{i=1}^{n} (f(U_i) - \alpha_n)^2}$$

- The method is more accurate as $\frac{\sigma_f}{\sqrt{n}}$ is reduced.

Motivation for Monte Carlo integration

Integrals can also be computed numerically with rules such as the trapezoidal rule.

$$\int_0^1 f(x)dx \approx \frac{f(0)+f(1)}{2n} + \frac{1}{n}\sum_{i=1}^{n-1} f(i/n)$$

There are more sophisticated rules for integration.

- The error on integration from the trapezoidal rule goes like $O(n^{-2/d})$ where d is the dimension of the integral.
- The error on Monte Carlo integration goes like $O(\frac{1}{(n^{1/2})})$ for n function values.
- The error from the Monte Carlo integration is independent of the dimension of the space.
- Monte Carlo methods are more efficient to compute multidimensional integrals than standard methods such as the trapezoidal rule.

Monte Carlo estimate of 1D integral

$$\int_0^1 (\cos(50x) + \sin(20x))^2 dx$$

```
myfun = @(x) ( cos(50 .*x) + sin(20 .*x) ).^2  ;
Nsample = 1024 ;

r = rand(Nsample , 1) ;
rf = myfun(r) ;
meanr = mean(rf) ;
err    = std(rf) / sqrt(Nsample ) ;

fprintf('Monte Carlo estimate (nsample=%d)  %f +/- %f \n' ,
    Nsample, meanr , err) ;
```

Monte Carlo estimate of 2D integral

$$\int_0^1 \int_0^1 (3x^2 + 5y^2) dx dy$$

```
fun = @(x,y) 3*x.^2 + 5*y.^2;
Nsample = 1024 ;
x = zeros(Nsample , 1) ;

for i=1:Nsample
  rx = rand() ;
  ry = rand() ;
  x(i) = fun(rx,ry) ;
end
meanr = mean(x) ;
err    = std(x) / sqrt(Nsample ) ;
fprintf('Monte Carlo estimate (nsample=%d)  %f +/- %f \n' ,
    Nsample, meanr , err) ;
```

Problems with the Monte Carlo method

It is easy to code the Monte Carlo method, but the method is slow! The error (err) with N_{sample} samples.

$$\text{err} \sim \frac{A}{\sqrt{N_{sample}}}$$

For example, the Monte Carlo estimate $N_{sample} = 1024$.

$$\int_0^1 (\cos(50x) + \sin(20x))^2 dx \approx 0.973 \pm 0.032$$

I estimate it would require $N_{sample} = 1 \times 10^{10}$ to get an error of 10^{-5}.
If the routines took 0.001 s, it would take 115 days to use $N_{sample} = 1 \times 10^{10}$.

Problems with the Monte Carlo method

$$\text{err} \sim \frac{A}{\sqrt{N_{sample}}}$$

There are a number of ways to reduce the N_{sample}. These are generically known as variance reduction methods, because the idea is to reduce A.

- importance sampling
- stratified sampling

The above methods will be tested on 1D integrals, even though we are mostly interested in the application of Monte Carlo methods to high dimensional integrals.

Timing of multi-dimensional integration

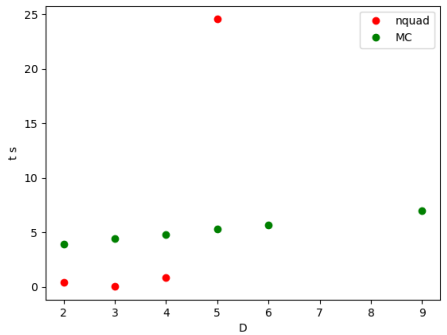
Consider the D-dimensional integral:

$$I(D) = \prod_{i=1}^{D} \int_0^1 dx^i \frac{1}{(1 + \underline{c}.\underline{x})^{D+1}}$$

where the two vectors with D components are defined as

$$\underline{c} = (0.2, ..., 0.2) \quad , \underline{x} = (x_1, x_2, ...x_D)$$

and $\underline{c}.\underline{x}$ is the dot product.
Time versus Dimension of the integral.



Monte Carlo integration

A more general formulae for Monte Carlo integration

$$\int_{-\infty}^{\infty} f(x)h(x)dx \approx \frac{1}{n}\sum_{i=0}^{n} f(x_i)$$

where x_i are random variables distributed according to the h(x) probability distribution.
The above equation assumes that the

$$\int_{-\infty}^{\infty} h(x)dx = 1$$

If $h(x)$ does not have the correct normalisation then, so that

$$\int_{-\infty}^{\infty} h(x)dx = V$$

$$\int_{-\infty}^{\infty} f(x)h(x)dx \approx V\frac{1}{n}\sum_{i=0}^{n} f(x_i)$$

Monte Carlo integration

The integral α

$$\alpha = \int_0^1 f(x)dx$$

can be interpreted as the expectation $E[f(U)]$ where U uniformly drawn from [0,1].

$$I_1 = \int_{-\infty}^{\infty} (x^2 + 3)e^{-x^2/2}dx$$

It is suggestive to use normal random numbers to simulate I_1.

$$I_2 = \int_0^{\infty} (1 + \sin(x))e^{-x}dx$$

It is suggestive to use random numbers distributed according the exponential to estimate I_2, but it may not be optimal.

Basic idea

Consider the integral below. The upper limit for the integral is L.

$$I(L) = \int_0^L e^{-x} dx$$

The integral can be estimated using Monte Carlo method

$$I(L) \approx \frac{L}{N} \sum_{i=1}^{N} e^{-y_i}$$

where y_i are random numbers uniformly distributed between 0 and L.
First consider L = 2 with 5 samples

$$\begin{aligned} I(2) &\equiv 2(e^{-0.161} + e^{-0.184} + e^{-1.1618} + e^{-0.913} + e^{-1.069}) \\ &= 2(0.8505 + 0.831 + 0.312 + 0.4011 + 0.343) \end{aligned}$$

The Monte Carlo estimate (exact 0.8646)

$$I(2) = 1.095849 \pm 0.241$$

Basic idea

Consider the integral below. The upper limit for the integral is L.

$$I(L) = \int_0^L e^{-x} dx = 1 - e^{-L}$$

where y_i are random numbers uniformly distributed between 0 and L.
Now consider L = 10 with 5 samples

$$\begin{aligned} I(10) &\equiv 10(e^{-1.411} + e^{-8.093} + e^{-2.76} + e^{-1.2377} + e^{-7.06}) \\ &= 10(0.243 + 0.00031 + 0.062 + 0.29 + 0.000857) \end{aligned}$$

The Monte Carlo estimate

$$I(10) = 1.20 \pm 0.62$$

compared against the exact value of 0.9999546.

Importance sampling

Now compute the integral using random numbers generated from the exponential probability distribution

$$I(L) = \int_0^L e^{-x} dx$$

Use random numbers (y_i) distributed from the exponential distribution. Consider L = 2 with 5 samples. My code gave the following random numbers from the exponential distribution:

$$1.41, 1.967, 0.360, 3.730, 1.31$$

$$I(2) \approx \frac{1}{5}(1 + 1 + 1 + 0 + 1)$$

The Monte Carlo estimate

$$I(2) = 0.8 \pm 0.2$$

compared against the exact value of 0.86466.

Importance sampling

Now compute the integral using random numbers generated from the exponential probability distribution

$$I(L) = \int_0^L e^{-x} dx$$

Use random numbers (y_i) distributed from the exponential distribution. Consider L = 10 with 5 samples. My code gave the following random numbers from the exponential distribution:

$$0.66, 1.63, 0.54, 0.0201, 0.701,$$

$$I(10) \approx \frac{1}{5}(1 + 1 + 1 + 1 + 1)$$

The Monte Carlo estimate

$$I(10) = 1.0 \pm 0.0$$

compared against the exact value of 0.999954.

Quick conclusions

- It is better if all the numbers $f(x_i)$ numbers are all the same magnitude, to reduce the variance
- One way is generate the random number from a "good probability distribution". This is known as importance sampling.
- Another way is to split the integration region into sub-intervals and apply the Monte Carlo integration to each integral individually.

Importance sampling

Normally you are just given an integral and you have to split the integrand into a probability part and the rest.

$$I = \int_b^a r(x)dx = \int_b^a h(x)f(x)dx$$

- Splitting $r(x)$ into two functions is not unique.
- It must be possible to generate random numbers from the $f(x)$.

The variance can be reduced if another probability distribution is used.

Importance sampling

The variance can be reduced if another probability distribution is used.

$$I = \int_b^a h(x)f(x)dx = \int_b^a \frac{h(x)}{s(x)} f(x)s(x)dx$$

where a different probability distribution is used in the integral.

$$E(\frac{h}{s}f) = \int_b^a \frac{h(x)f(x)}{s(x)} s(x)dx$$

$$\int_b^a h(x)f(x)dx \approx \frac{1}{n}\sum_{i=1}^{n} \frac{h(y_i)f(y_i)}{s(y_i)}$$

where y_i are random variables distributed according to the s distribution.

Example of importance sampling

$$I = \int_0^1 \frac{e^{-x}}{1+x^2} dx$$

Try three new probability distributions in importance sampling.

$$\begin{aligned} f_0(x) &= 1 \quad 0 < x < 1 \\ f_1(x) &= e^{-x} \quad 0 < x < \infty \\ f_4(x) &= 4(1+x^2)^{-1}/\pi \quad 0 < x < 1 \end{aligned}$$

The above functions are chosen to try and make the non-probability part of the integrand flat to reduce the variance.

Importance sampling via exponential random

```
myfunA = @(x) ( 1.0  ./ (1 + x.^2  ))  ;
rexp = exprnd(1,Nsample , 1 ) ;
rA = myfunA(rexp) ;
%% enforce  x <= 1
for i = 1:Nsample
   if( rexp(i) > 1 )
      rA(i) = 0.0 ;
   end
end
meanrA = mean(rA) ;
errA    = std(rA) / sqrt(Nsample ) ;
fprintf('MC (exp numbers) estimate =  %f +/- %f \n' ,  meanrA
    , errA ) ;
```

Importance sampling with $1/(1+x^2)$

```
rC = rand(Nsample , 1) ;
rCtrans = tan( pi .* rC ./ 4 ) ;
myfunC = @(x) ( myfun(x) .* (1 + x.^2  ) .* pi ./ 4  )  ;
aaa = myfunC(rCtrans) ;
for i = 1:Nsample
   if( rCtrans(i) > 1 )
      aaa(i) = 0.0 ;
   end
end
meanrC = mean(aaa) ;
errC   = std(aaa) / sqrt(Nsample ) ;
fprintf('MC (1+x**2) estimate =  %f +/- %f \n' ,  meanrC ,
    errC ) ;
```

Output from importance sampling example

```
Integral from Matlab function = 0.524797
MC estimate (nsample=1024)    =  0.520 +/- 0.008
MC (exp numbers) estimate     =  0.52  +/- 0.01
MC (1+x*2) estimate           =  0.536 +/- 0.004
```

- The most efficient method has the smallest variance.
- As usual in my experience it takes a couple of trials to find a new probability distribution which gives a smaller variance.
- The best method with random numbers $\frac{1}{(1+x^2)}$.

Introduction to stratified sampling

We wish to integrate the following integral by Monte Carlo

$$I = \int_a^b f(x)dx$$

The key quantity for the Monte Carlo integration is the variance

$$\sigma_I^2 = \int_a^b (I - \overline{f})^2 dx$$

The integral can also be written as

$$I = I_1 + I_2$$

$$I_1 = \int_a^\alpha f(x)dx \quad I_2 = \int_\alpha^b f(x)dx$$

$$\sigma_{I_1}^2 = \int_a^\alpha (I_1 - f(x))^2 dx \quad \sigma_{I_2}^2 = \int_\alpha^b (I_2 - f(x))^2 dx$$

Stratified sampling cont.

Consider the Monte Carlo estimate of using random variables x_i distributed between a and b

$$I = \int_a^b f(x)dx \approx (b-a)\frac{1}{N}\sum_{i=0}^{N} f(x_i)$$

$$I_1 + I_2 = \int_a^{\alpha} f(x)dx + \int_{\alpha}^{b} f(x)dx$$

$$I_1 + I_2 \approx V_1\frac{1}{N_1}\sum_{i=0}^{N_1} f(y_i) = V_2\frac{1}{N_2}\sum_{i=0}^{N_2} f(z_i)$$

where y_i (z_i) are uniformly distributed between a and α (α and b).

- The variance $\sigma_{I_1+I_2}$ can be made less than σ_I w

A numerical test

Try stratified sampling on a simple integral.

$$f(x) = \frac{e^{-x}}{(1+x^2)}$$

Compare the Monte Carlo estimate of

$$I = \int_0^1 f(x)dx$$

to

$$I = \int_0^{1/4} f(x)dx + \int_{1/4}^{1/2} f(x)dx + \int_{1/2}^{3/4} f(x)dx + \int_{3/4}^{1} f(x)dx$$

Sample matlab code

```
Nsample = 1024 ; Ncopy = 10 ;
xxx = zeros(Ncopy,1)  ;
for ic=1:Ncopy
  r1 = 0.25 * rand(Nsample/4 , 1) ;
  r2 = 0.25 + 0.25 * rand(Nsample/4 , 1) ;
  r3 = 0.5 + 0.25 * rand(Nsample/4 , 1) ;
  r4 = 0.75 + 0.25 * rand(Nsample/4 , 1) ;
  r1f = myfun(r1) ; r2f = myfun(r2) ;
  r3f = myfun(r3) ; r4f = myfun(r4) ;
  xxx(ic) = 0.25*(mean(r1f) + mean(r2f) + mean(r3f)  + mean(
      r4f) )  ;
end
xm  = mean(xxx)  ;  xstd = std(xxx) ;
fprintf('4-Stratified MC estimate (nsample=%d, Ncopy=%d)  %f
   +/- %f \n' , Nsample, Ncopy, xm , xstd) ;
```

Result of the numerical tests

A numerical experiment produced

```
Integral from Matlab integral function   = 0.524797
MC (nsample=1024)                        = 0.533 +/- 0.008
MC (nsample=1024, Ncopy=10)              = 0.52  +/- 0.01
2-Stratified MC (nsample=1024, Ncopy=10) = 0.526 +/- 0.002
4-Stratified MC (nsample=1024, Ncopy=10) = 0.525 +/- 0.002
```

$$\frac{0.01}{0.002} = 5$$

The errors go like $\frac{1}{N_{sample}}$, so $(5)^2 \sim 25$ fewer samples would be required.

- It can be useful to do different split up of the integand.

Adaptive Multidimensional Integration

- You will probably have noticed that Monte Carlo integration involves a lot of freedom to choose techniques, such as importance sampling and/or stratified sampling.
- It does seem that the computer could do more of the work by measuring and then tuning.
- One of the standard multi-dimensional integration algorithms is called VEGAS and was developed by Peter Lepage.

```
A New Algorithm for Adaptive Multidimensional Integration
G.Peter Lepage (SLAC). Apr 1977. 19 pp.
Published in J.Comput.Phys. 27 (1978) 192
Detailed record - Cited by 865 records
```

Monte Carlo method in Statistical Physics

- The Monte Carlo method is extensively used to simulate statistical mechanics.
- For example, the Ising model, which is a simple model of a magnet.
- Researchers are interested in phase transitions, such as water being either liquid, solid, or gas at different temperatures.

Percolation theory

Consider a square lattice with a filling probability p_p

- Loop through the sites of the lattice.
- At each site generate a uniform random number r, if $r < p_p$ then fill the square.

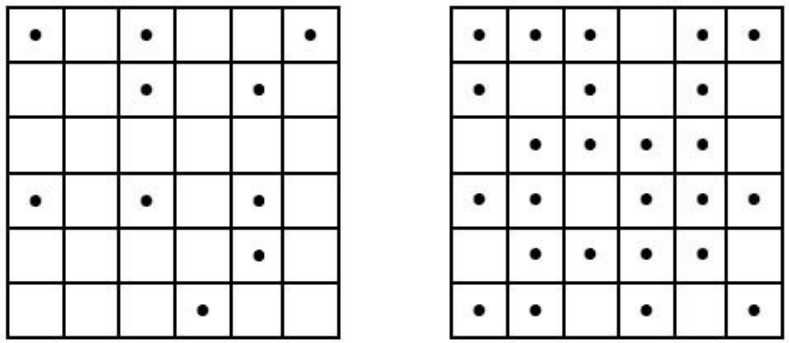


Figure 2: Percolation on a square lattice for two percolation probabilities: (a) low probability; (b) high probability

Percolation theory

An ensemble of lattices is generated

- Each lattice is analyzed to look for "spanning paths."
- At a certain critical filling probability p_p there will be spanning paths that connect sides of the square.
- The results depend on the size of the square.

For a square lattice

$$p_p = 0.59274621 \pm 0.00000013$$

There are arguments that these simple models share properties with more realistic systems.

Applications of percolation theory

Some applications of percolation theory

- Spread of infectious disease.
- Spread of forest fires. There is a probability that fire spreads from tree to tree. When there are spanning paths, the forest burns down.
- Spread of terrorism.

Using Monte Carlo methods to study gambling

- The Monte Carlo method can be used to study gambling problems.
- Some of the problems can solved analytically, but the Monte Carlo is a simple way to study the problem.

For example two players A and B start with stakes of £10 and £20 respectively. A single game is played by selecting a random number r

- if $r < 0.5$ A wins, A gets £1 from B.
- if $r > 0.5$ B wins, B gets £1 from A

The game is terminated when either A or B runs out of money. By repeating games and counting outcomes, probabilities can be assigned to various strategies.

Matlab code for simulation of a single game

```
Astake = 10 ; Bstake = 20 ;
count = 0 ;
while( Astake > 0 && Bstake > 0 )
  rr = rand()  ;
  if(  rr < 0.5 )
    Astake = Astake + 1 ;
    Bstake = Bstake - 1 ;
  else
    Astake = Astake - 1 ;
    Bstake = Bstake + 1 ;
  end
count = count +  1 ;
end
fprintf('A stake = %d A stake = %d length of game = %d\n',
    Astake, Bstake,count)
```

Monte Carlo simulation of gambling

- In the previous slide I showed Matlab code to simulate one game.
- An outer loop is added so that the games are repeated 1000 times.
- The number of times A wins and the number of times B wins is recorded. This gives the probability of A and B winning.
- For example, if A wins 357 times out of 1000. The probability of A winning is 0.357.

Results for gambling

The output from my code with multiple games.

```
Initial stake of A = 10 of B = 20
Number of copies = 1000
Length of games = 199.1 +/- 5.7
Prob A to win = 0.357 Prob B to win = 0.643
```

- Various "wining strategies" can be investigated.
- For this game, the winner has the larger initial amount of money.
- The martingale strategy, where a gambler doubles their stake every time they lose, can be investigated.

Randomness

In Matlab a random number distributed between 0 and 1 can be generated by:

```
> rand()

ans =

    0.8147
```

- The **rand(n,1)** generates n random numbers randomly distributed between 0 and 1.
- But what does the function actually do?

Pseudo-random numbers

- Computers are normally deterministic.
- The random numbers generated by the computer are actually pseudo-random numbers, because they are generated by a deterministic algorithm, but just appear random.
- In nature true random numbers are generated by nuclear decays. Tables of random numbers are made available, but these are not practicable for general work. See `http://www.fourmilab.ch/hotbits/` for example.
- Need to generate random numbers quickly, so solving chaotic systems on the computer are of no use.

Basic random number generator

Recall

- 7 mod 5 = 2
- 10 mod 5 = 0
- 43 mod 5 = 3

The most basic random number generator is the **linear congruential generator**. Consider the sequence

$$x_{i+1} = ax_i \bmod m$$

where a is an integer known as the multiplier and m is the modulus.

- x_i lie between 0 and m-1
- $u_i = x_i/m$ is a candidate for a variable uniformly distributed between 0 and 1
- The first number in the sequence (x_0) is known as the **seed**.

Example random number generation

Consider

$$x_{i+1} = ax_i \bmod m$$

with a = 6 and m = 11 (in practice much bigger values would be used.)

- Use the see $x_0 = 1$.
- Then the sequence is generated : 1 , 6 , 3 , 7 , 9 , 10 , 5 , 8 , 4 ,2 , 1, 6
- Recall the pseudo-random number is the above numbers divided by 11 to get numbers between 0 and 1.
- Once the seed is reproduced, the sequence repeats itself (and the numbers are no longer random).
- The number of elements in a cycle is the period.
- The maximum period is m-1 (known as a full period).

Seeds and random numbers

- The linear congruential generator is combined with other techniques to produce a good quality random number generator.
- You should always use the standard random number generator (Rather than write your own).
- There is usually a **seed** function which controls the place in the random sequence.

```
>> rng(100) ;
>> rand(3,1)
ans = 0.5434 0.2784 0.4245

>> rng(100) ;
>> rand(3,1)
ans = 0.5434 0.2784 0.4245
```

Random numbers from different distributions

- Many applications require random numbers generated from different probability distributions (for example (exponential, normal distribution,...)
- First random numbers are generated from a uniform distribution and then a function is applied to get random variables distributed with the required probability distribution.

The common methods for generating random numbers with various probability distributions are:

- Transformation method
- Rejection method.
- (NEW) Normalizing flow.

To generate random variables distributed with the normal distribution.

To generate random variables distributed with the following distribution

$$p(y)dy = \frac{1}{\sqrt{2\pi}} e^{-y^2/2} dy$$

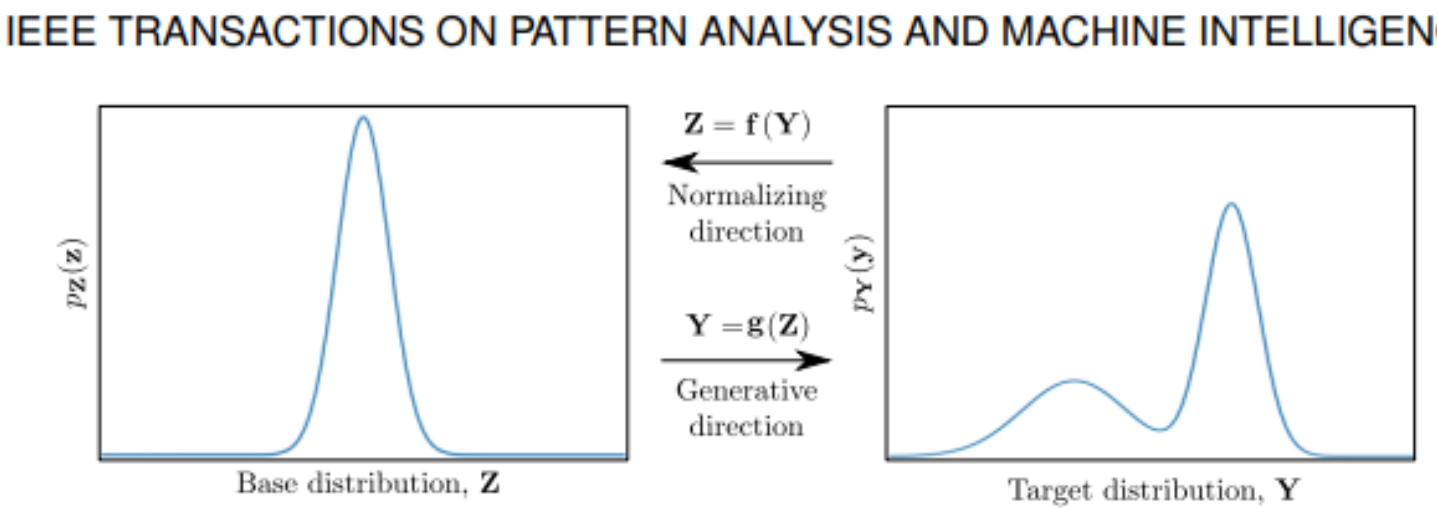
consider random variables x_1 and x_2 uniformly distributed between 0 and 1.

$$\begin{aligned} y_1 &= \sqrt{-2\log x_1}\cos(2\pi x_2) \\ y_2 &= \sqrt{-2\log x_1}\sin(2\pi x_2) \end{aligned}$$

- y_1 and y_2 are distributed with the normal distribution.
- This is known as the Box-Muller method.

Generating flow

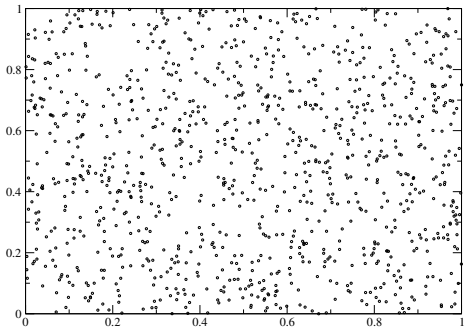
Normalizing Flows are generative models which produce tractable distributions where both sampling and density evaluation can be efficient and exact.

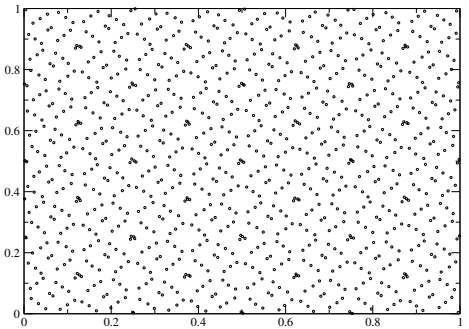


New machine learning / AI based method.
`https://arxiv.org/abs/1908.09257`

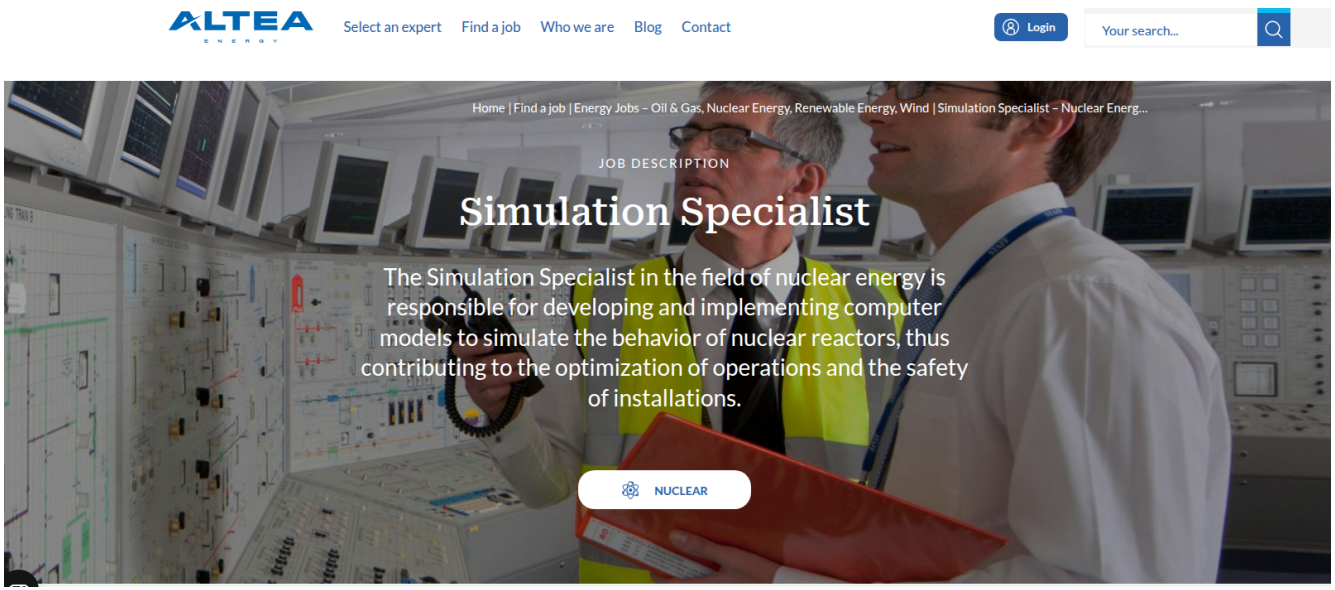
Introduction to quasi-random numbers

- Quasi-random numbers are applied to numerical integration.
- Points below can be used in two dimensional integrals.
- For small dimensions the cost of quasi-random integration is $O(1/N_{samp})$ (better than MC).
- Plot pseudo-random numbers on the left and the quasi-random (Sobol) numbers on the right.





Neutron scattering



- An example of solving a Partial Differential Equation (PDE) using the Monte Carlo Method.
- This was one of the original applications of the Monte Carlo method.

Slides from Robert Jack, SAMBa ITT, June 2015.

Neutron Transport

- Monte Carlo for the diffusion equation and some relatives (simulating random walks, which obey diffusion-like equations)
- Monte Carlo for the Neutron Transport Equation (simulating the motion of “fictitious” neutrons)

Monte Carlo diffusion

Numerical solution of PDEs…

"Density" $\rho = \rho(x,t), \qquad x \in \mathbb{R}^d, \qquad t \geq 0$

Solve $\partial_t \rho = \triangle \rho$ given $\rho(x,0)$

Or, solve $\partial_t \rho = \nabla \cdot (\nabla \rho - F\rho)$ given $\rho(x,0)$ and $F = F(x)$

[Maybe you know… for a particle obeying a stochastic differential equation

$$\mathrm{d}x = F\mathrm{d}t + \sqrt{2}\,\mathrm{d}B_t$$

… the probability density function ρ for its position solves the PDE above (Fokker-Planck equation)]

Assume normalisation $\int \rho(x,0)\mathrm{d}x = 1$ [equation is linear]

Monte Carlo diffusion

$$\partial_t \rho = \nabla \cdot (\nabla \rho - F\rho) \qquad \mathrm{d}x = F\mathrm{d}t + \sqrt{2}\,\mathrm{d}B_t$$

A possible method for solving this equation:

Initialise a particle at some point x_0, set time $t = 0$

Move it to x_0+Δ with $\Delta = Fh + B$, where B is Gaussian distributed with variance $2h$. Increment time by h

Repeat until time reaches end of required interval

Repeat the whole thing many times, with initial x_0 distributed as $\rho(x,0)$

Can prove:

As $h \to 0$, can estimate $\int f(x)\rho(x,t)\mathrm{d}x$ as $\frac{1}{N}\sum_i f(x_i(t))$

(The sum runs over the N simulated particles)

Estimator is unbiased in the limit, variance of estimator ~ $(1 / N)$

Can also show: many similar methods are possible and also work…

Neutron Transport

- Think of the MC method as a simulation of a fictitious process that obeys the neutron transport equation (NTE) Proving that the fictitious process obeys the NTE should be the same as the derivation of the NTE. . .
- This means that whatever quantities are relevant for the neutrons can be evaluated directly from the fictitious process.

Additional physics that needs to be included.

- Neutrons have velocities (advective terms in PDE), and they scatter (scattering terms).
- Neutrons appear from sources and fission (source terms) disappear (from absorption) **particles not conserved.**

Discrete event simulation

- A discrete-event simulation (DES) models the operation of a system as a (discrete) sequence of events in time.
- Each event occurs at a particular instant in time and marks a change of state in the system.
- One application is modeling a queue, such as a patients entering a Hospital.
- Random numbers are used to model the stochastic nature of the patients entering the hospital. This is the connection to the Monte Carlo method.

The computer model allows you to test different arrangements of the queue without having to implement and test them in the real world (which can be expensive.) Part of a digital twin model.

Introduction

Queuing theory is a branch of operational research concerned with problems involving queuing or waiting.

Examples of queues

- *Parking place*. They are going to make a new parking place in front of a supermarket. How large should it be?

- *Production system*. A machine produces various types of products. What is the production lead time of an order? What is the reduction in the lead time when we have an extra machine? Should we assign priorities to the orders?

- *Data communication*. In computer communication networks standard packages called cells are transmitted over links from one switch to the next. In each switch incoming cells can be buffered when the incoming demand exceeds the link capacity. Once the buffer is full incoming cells will be lost. What is the cell delay at the switches? What is the fraction of cells that will be lost? What is a good size of the buffer?

- Queues form because resources are limited.

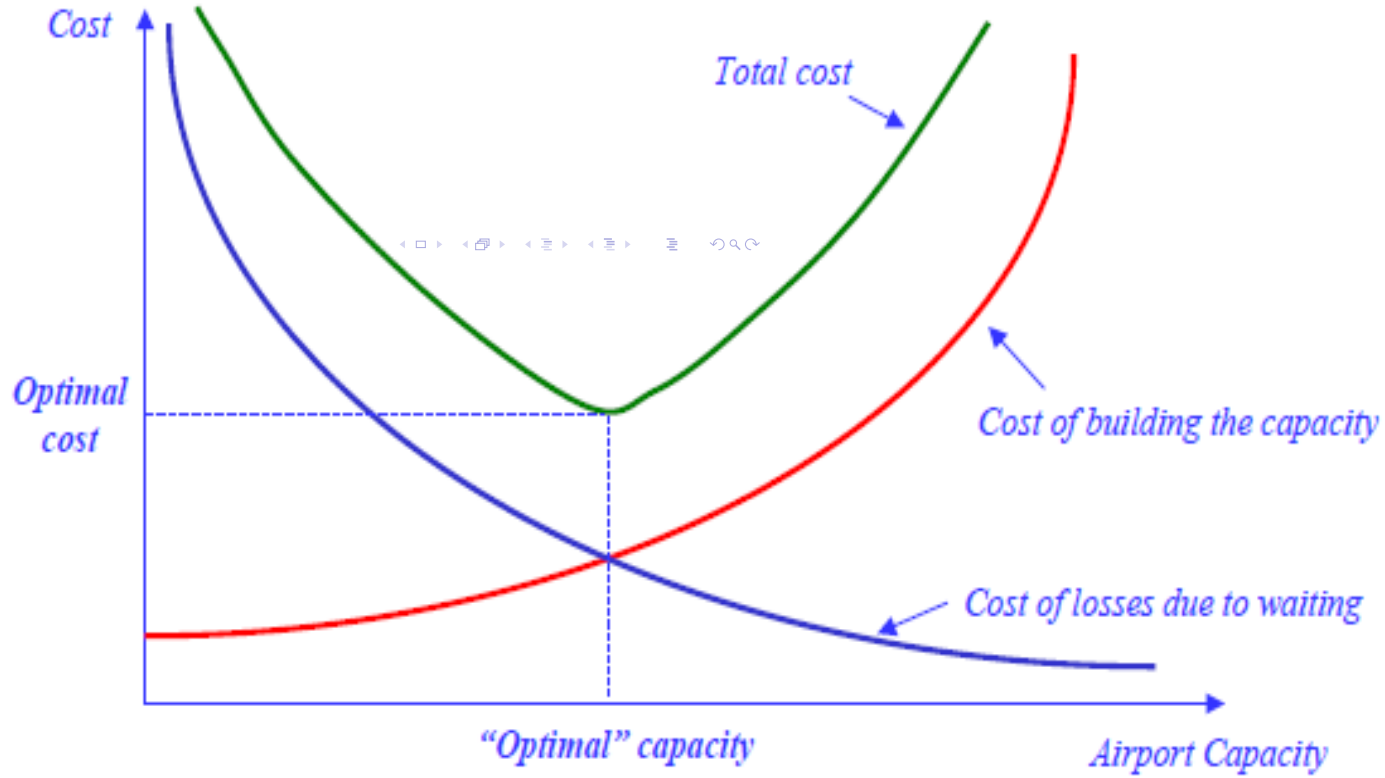
- It makes economic sense to have queues. Providing too much service involves excessive costs (how many trains/buses would be needed to eliminate waiting? How many supermarket tills would be needed to avoid queuing?).

- We aim for a balance between service to customers (not providing enough service capacity causes the waiting line to become excessively long) and economic considerations.

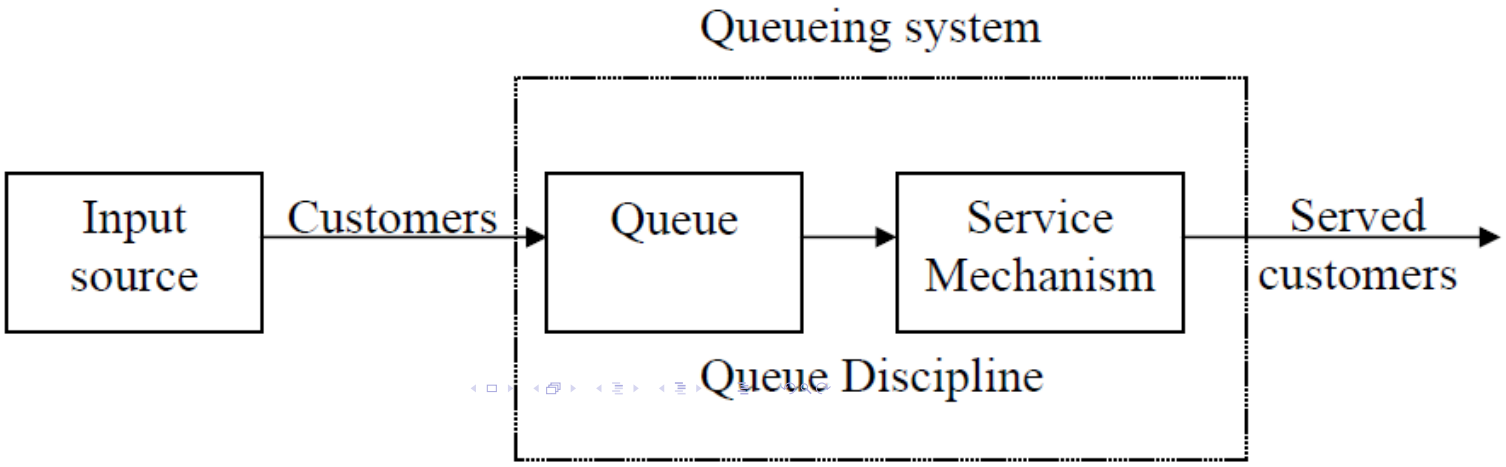
- Thus, queuing theory is applied in economic analyses involving trade-offs among operating costs, capital investments and level-of-service standards (balance between cost of service and cost associated with waiting for that service).

Queuing models can be essential in analysis of capital investments

Example (effectiveness against cost):



Basic structure of a queuing model



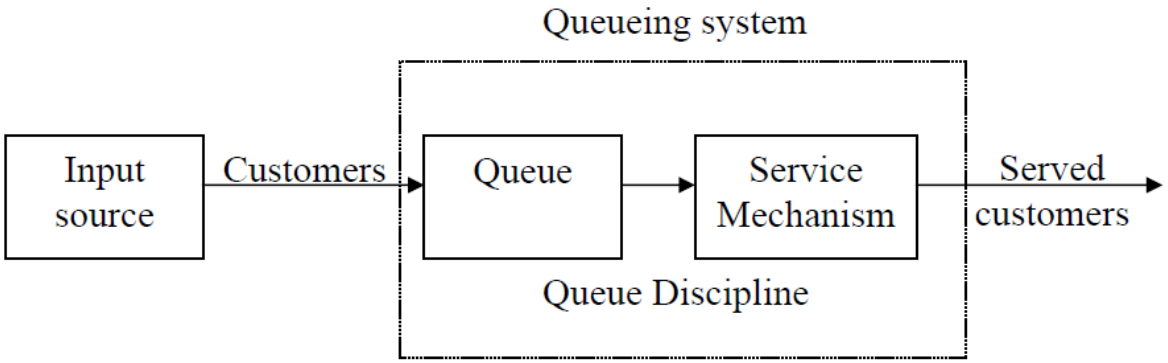
Note:

1. customers need not to be people (they can be also parts, vehicles, machines, jobs);

2. queue might not be a physical line (for example: customers on hold, jobs waiting to be printed, planes circling above an airport).

Components of the model

- Input source
- Arrival process (demand rate)
- Queue characteristics (queue capacity, queue discipline)
- Service mechanism (service times)



Arrival process

- How customers arrive, e.g. singly or in groups?
- When will each one arrive, i.e. how the arrivals are distributed in time? Associated with a probability distribution - usually Poisson distribution or exponential distribution.
- Customers may be *balking* - refuse to enter the system if the queue is too long (also spelled as *baulking*).

Queue characteristics: queue capacity

- A queue is characterized by the maximum permissible number of customers that it can contain. Queue capacity may be infinite or finite.
- Also called as *buffer size*.

Queue characteristics: queue discipline

Refers to the order in which members of the queue are selected for service.

- First-come-first-served.
- Last-come-first-served.
- Request-dependent, e.g. quick/easy jobs first/last.
- According to request priority.
- Randomly.
- Simultaneously.

Other queue characteristics

- *Balking* (or *baulking*) – customers deciding not to join the queue if it is too long.
- *Reneging* – customers leave the queue if they have waited too long for service.
- *Jockeying* – customers switch between queues if they think they will get served faster by doing so.

www.youtube.com/watch?v=PiOi4ZqAFKE

Typical performance questions

What is the…

- average waiting time in the queue?
- average number of customers in the queue?
- average time a customer spends in the system?
- probability a customer is rejected?
- fraction of time a server is idle?

In queuing theory, analyses take form of asking „**what if**” questions rather than trying to optimise the design. However, analyses are used to *support* decision-making.

Probability distributions

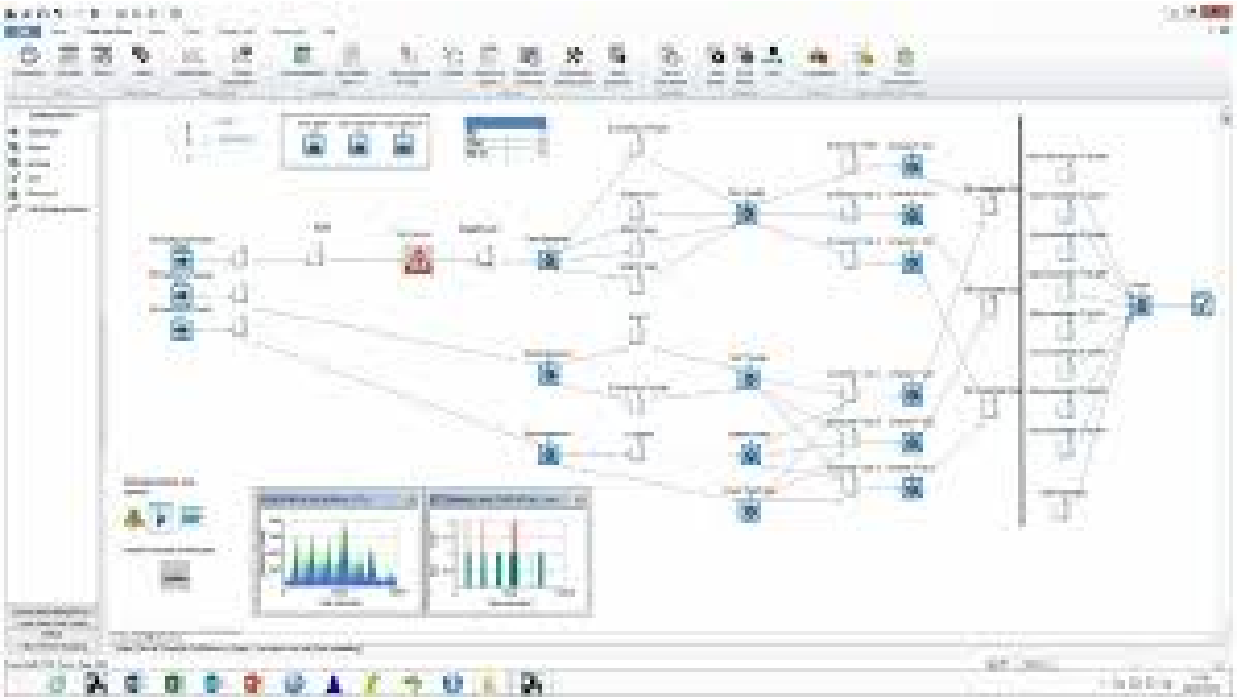
The patients entering a hospital arrive at random interval that is defined by a probability distribution.
Example probability distributions.

- Poisson distribution.
- Exponential distribution or Erlang distribution.
- Or use measured histograms of the real system to generate the probability distribution.

Introduction to SIMUL8

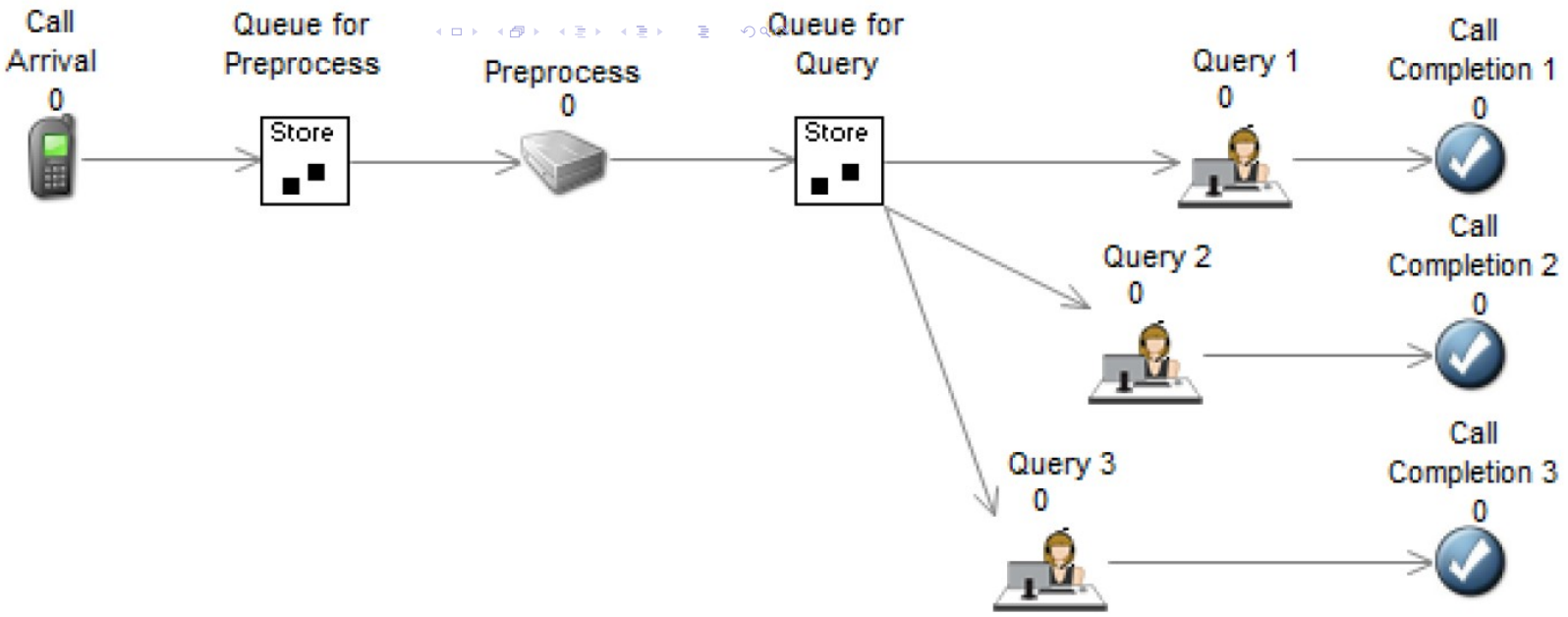
- SIMUL8 software is used for simulating systems that involve processing of discrete entities at discrete times.
- SIMUL8 allows its user to create a computer model, which takes into account real life constraints, capacities, failure rates, shift patterns, and other factors affecting the total performance and efficiency of production.
- SIMUL8 has a graphical user interface. It is possible to also add custom routines using a visual basic like language.
- SIMUL8 is installed on the University computers.
- SIMUL8 runs on Windows or in the cloud.

Example of SIMUL8 GUI



Prioritizing customers

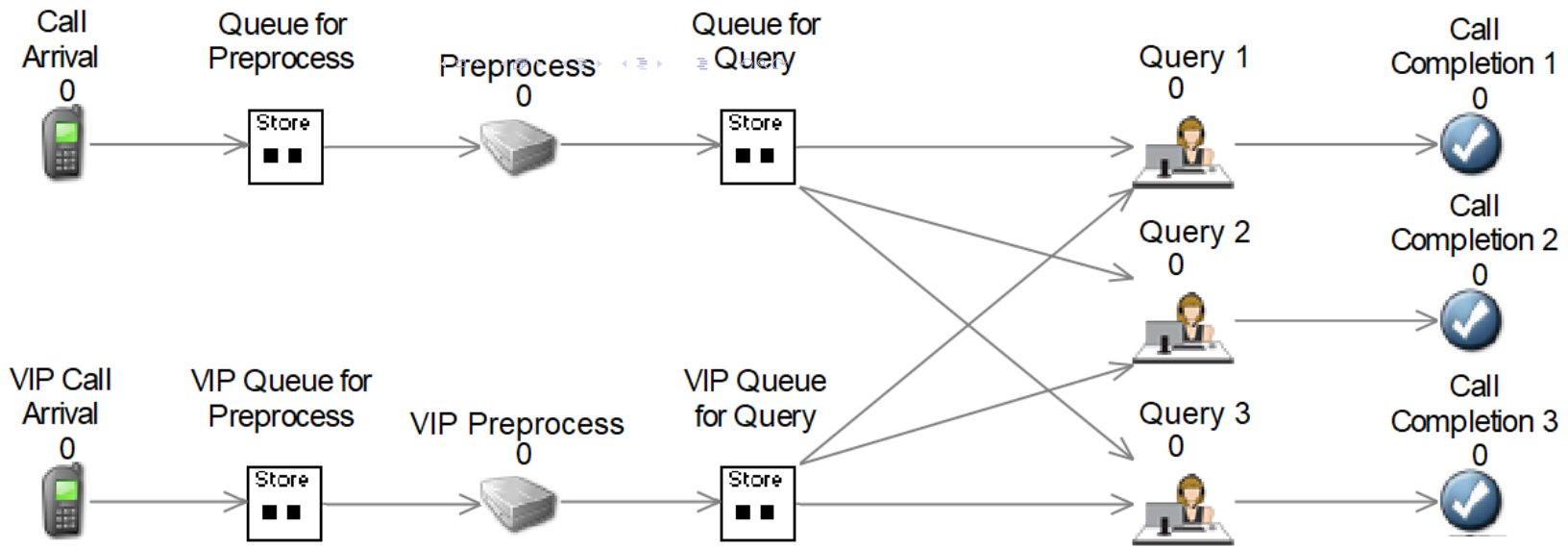
- Priority customers - do not need to wait in a queue with everyone else.
- How to model that?



Above: Call3.S8

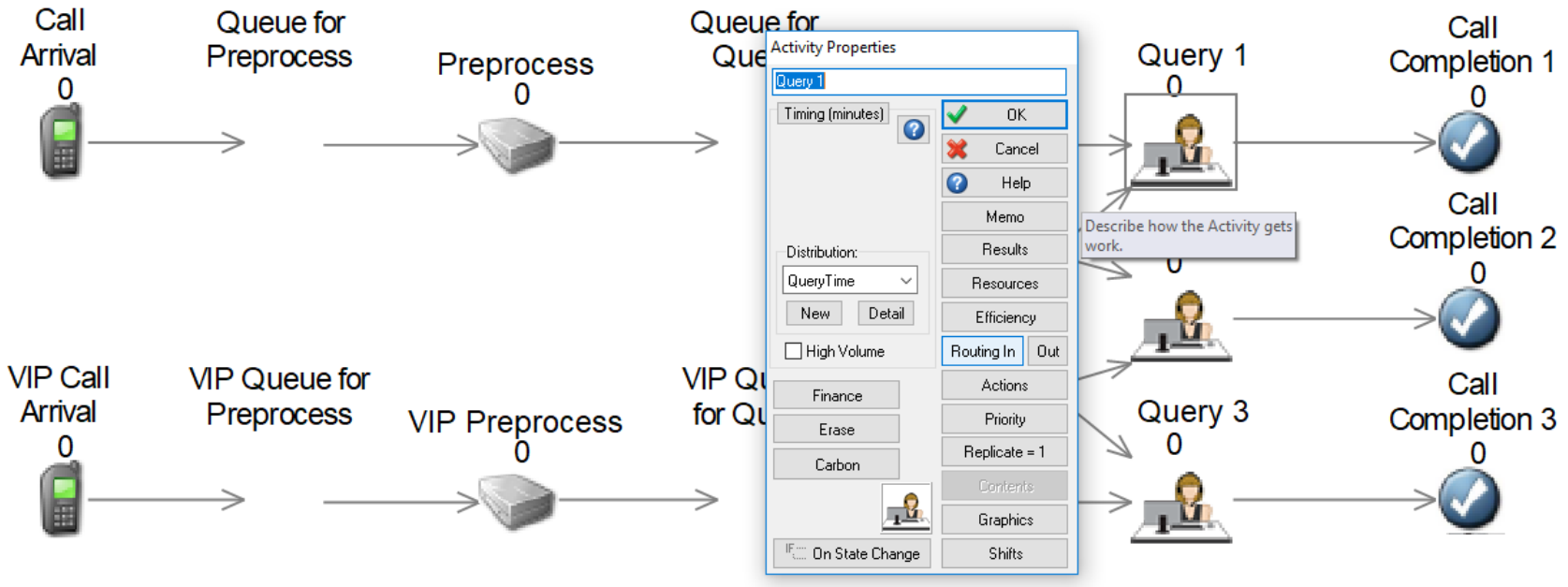
Prioritizing customers

- Step 1: we create a separate entry point for priority callers.



Prioritizing customers

- Step 2: change the priority for routing into the three operators activities:
  - double click on Query 1
  - Select "**Routing In**"
  - Set the Discipline to Priority (repeat for Query 2 and 3)

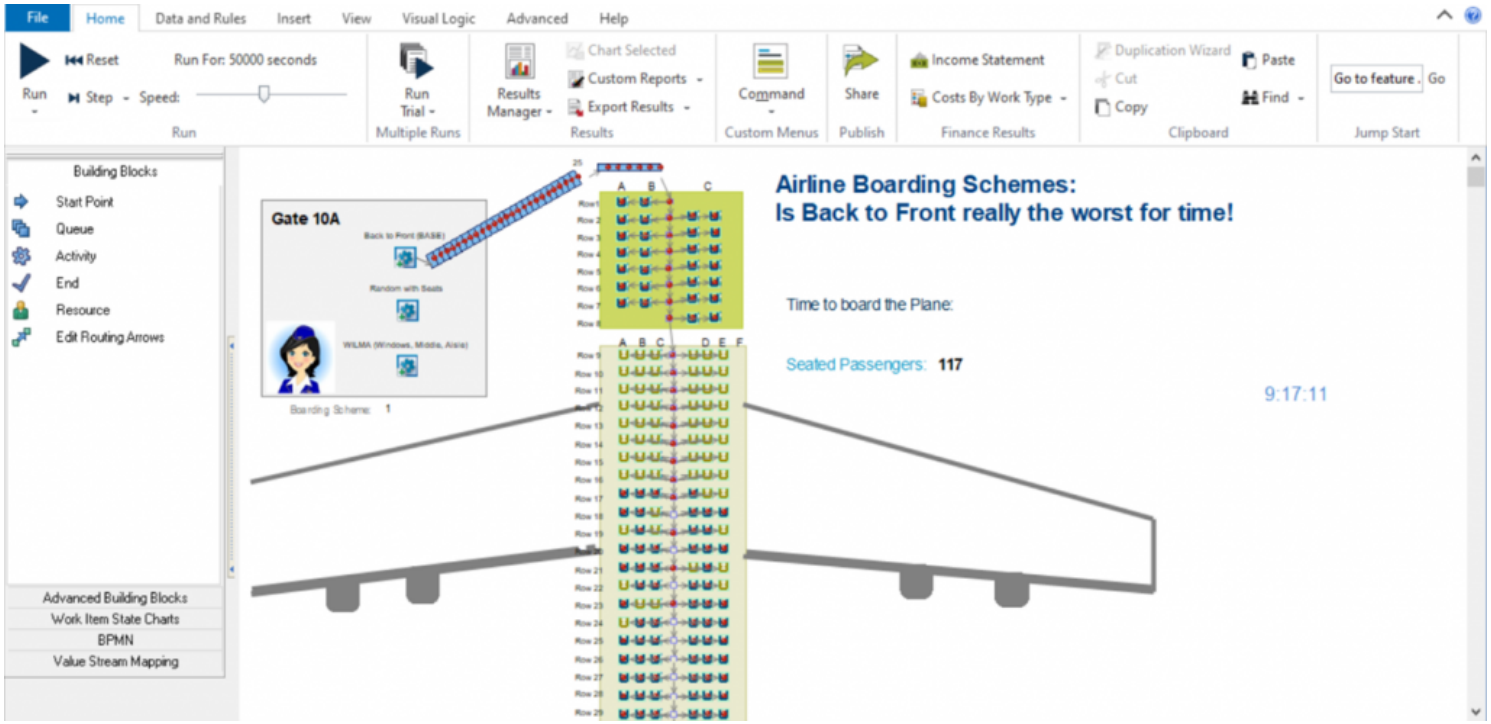


Manufacturing Assembly Line Industry Example - SIMUL8

https://youtu.be/L18QGE6VQfY?si=XCLXC_3qgTfm5cNh

What is the quickest way to board an airplane?

Test different ways of passangers boarding an airplane.



What is the quickest way to board an airplane?

Method	Description	Time (minutes)
Back-to-Front	Business boarded first, then the zones were boarded starting in the back and moving to the front of the plane.	25.07
Random	Business boarded first, then all rows and all passengers were allowed to board.	21.09
WILMA	Business boarded first, followed by all window seats, then all middle seats, then all aisle seats.	17.47

Critical issue for budget airline.

Alternative software

The University of Plymouth has an educational license for SIMUL8. (I am the contact for the University)

Other software:

- Matlab has simulink `https://uk.mathworks.com/solutions/discrete-event-simulation.html`
- Python has a library called simpy that does discrete event simulation `https://simpy.readthedocs.io/en/latest/`.

Some bigger integrals ....

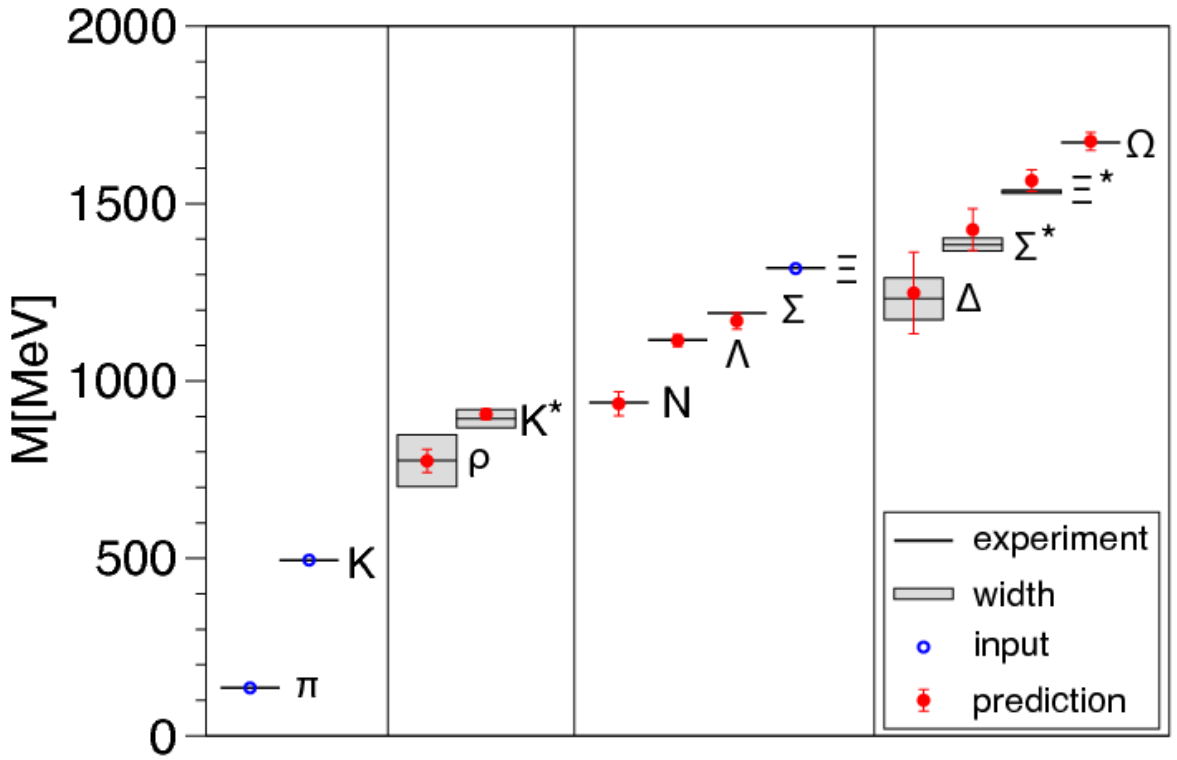
The theory of nuclear physics can be written as a large multi-dimensional integral.

$$c_{ij}(t) = \frac{1}{Z} \int du \int d\psi \overline{d\psi} O(t)_i O(0)^\dagger_j e^{-S_F - S_G}$$

- Quantum mechanics and quantum field theory can be written in terms of "path integrals."
- Nuclear Physics and materials systems can be written in the above form.
- For QCD, the dynamics of the gluon and quarks is in the Dirac action (S_F) and gauge action (S_G).
- The multi-dimensional integral is computed in using Monte Carlo techniques on the computer.
- Dimensions $> 10^6$

Results from lattice field theory

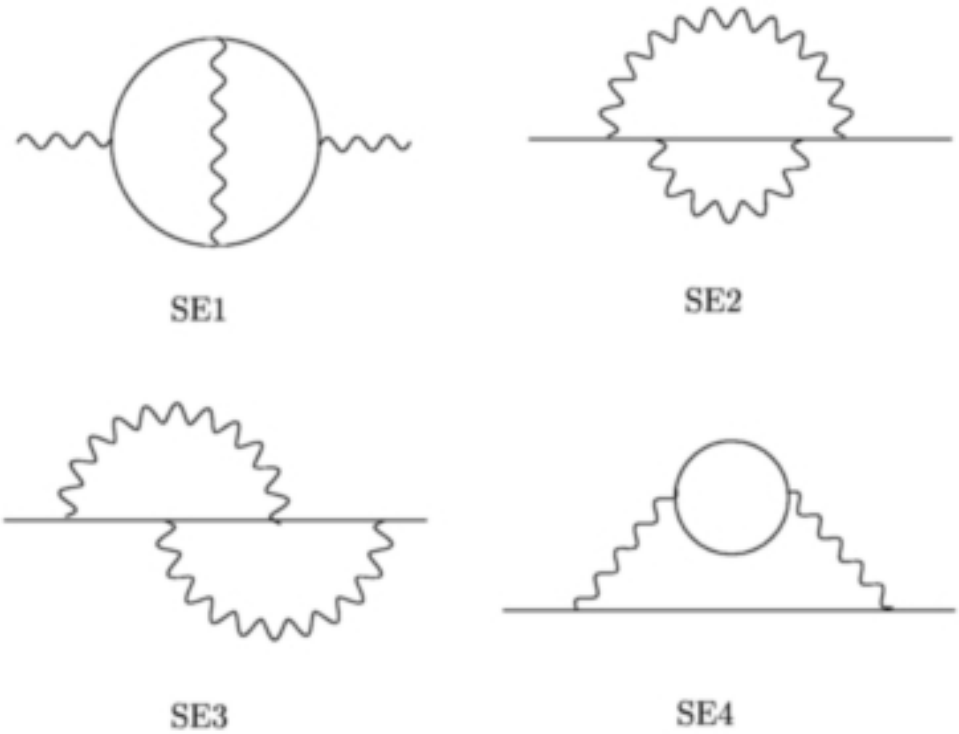
N is the proton that is the basic building block of us.



Quantum Field Theory

- Feynman diagram are a way to solve quntum field theory using perturbation theory (an expansion in small parameter.)

The Feynman diagrams below are actually 8 dimensional integrals.



I believe 20 dimensional integrams are used in state of the art calculations.

Summary of Monte Carlo methods

- Monte Carlo methods can solve sytems in many dimensions.
- Monte Carlo methods produce low accuracy solutions.
- Various tricks such variance reduction are required to reduce the errors.

Summary of the lecture

- Random variable generation.
- Monte Carlo integration,
- Variance reduction techniques:
  - importance sampling.
  - Stratified sampling
- Simulation of Physics models.
- Simulation of gambling.
- Monte Carlo simulations of Physical Systems.
- Generation of random numbers.
- Solving a Partial Differential Equation (PDE) using MC.
- Queuing systems.